

Project 3

May 4, 2025

0.1 Load in packages

```
[1]: pip install pandas
```

```
Requirement already satisfied: pandas in /srv/conda/lib/python3.11/site-packages (2.2.3)
Requirement already satisfied: numpy>=1.23.2 in /srv/conda/lib/python3.11/site-packages (from pandas) (2.2.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /srv/conda/lib/python3.11/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /srv/conda/lib/python3.11/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /srv/conda/lib/python3.11/site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /srv/conda/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import statsmodels.api as sm
plt.style.use('fivethirtyeight')
import seaborn as sns
from tqdm import tqdm
import xarray as xr
from sklearn.utils import resample
import statsmodels.formula.api as smf
import statsmodels.api as sm
```

0.1.1 Load in relevant datasets

```
[3]: climconflict = pd.read_csv('Data 148 Project 3 Data/clim_conflict_for_R.csv')
climconflict
```

```
[3]:
```

	country	countryisocode	year_actual	ccode	war_prio	any_prio	\
0	Guinea-Bissau	GNB	1981	404	0	0	

1	Guinea-Bissau	GNB	1982	404	0	0
2	Guinea-Bissau	GNB	1983	404	0	0
3	Guinea-Bissau	GNB	1984	404	0	0
4	Guinea-Bissau	GNB	1985	404	0	0
...	...	...	...	...	...	...
1044	Sudan	SDN	2002	625	1	1
1045	Sudan	SDN	2003	625	1	1
1046	Sudan	SDN	2004	625	1	1
1047	Sudan	SDN	2005	625	0	1
1048	Sudan	SDN	2006	625	1	1

	war_prio_on	any_prio_on	gdp	gdp_l	...	cru_prec_diftrend	\
0	0.0	0.0	647.1996	521.2217	...	0.261924	
1	0.0	0.0	586.6083	647.1996	...	0.070554	
2	0.0	0.0	636.3018	586.6083	...	-0.371771	
3	0.0	0.0	571.7829	636.3018	...	0.018248	
4	0.0	0.0	505.2217	571.7829	...	-0.035366	
...	...	...	...	...	...	...	...
1044	NaN	NaN	1122.9100	1094.5720	...	-0.074588	
1045	NaN	NaN	1176.4220	1122.9100	...	NaN	
1046	NaN	NaN	NaN	1176.4220	...	NaN	
1047	NaN	NaN	NaN	NaN	...	NaN	
1048	1.0	NaN	NaN	NaN	...	NaN	

	cru_prec_diftrend_lag	gdpcapgrowth	gdpcapgrowth_lag	\
0	-0.399240	24.169734	NaN	
1	0.261924	-9.362073	24.169734	
2	0.070554	8.471326	-9.362073	
3	-0.371771	-10.139670	8.471326	
4	0.018248	-11.640992	-10.139670	
...	...	...	...	...
1044	-0.011706	2.588957	4.472503	
1045	NaN	4.765475	2.588957	
1046	NaN	NaN	4.765475	
1047	NaN	NaN	NaN	
1048	NaN	NaN	NaN	

	gdpcapgrowth_lag2	gdpcap1980	polity2	polity2_lag	polity2_lag2	\
0	NaN	521.2217	-7	-7	-7.0	
1	NaN	521.2217	-7	-7	-7.0	
2	24.169734	521.2217	-7	-7	-7.0	
3	-9.362073	521.2217	-8	-7	-7.0	
4	8.471326	521.2217	-8	-8	-7.0	
...	...	...	...	...	...	...
1044	7.004418	1066.4200	-6	-7	-7.0	
1045	4.472503	1066.4200	-6	-6	-7.0	
1046	2.588957	1066.4200	-6	-6	-6.0	

1047	4.765475	1066.4200	-4	-6	-6.0
1048	NaN	1066.4200	-4	-4	-6.0

	polity1980
0	-7
1	-7
2	-7
3	-7
4	-7
...	...
1044	-7
1045	-7
1046	-7
1047	-7
1048	-7

[1049 rows x 230 columns]

```
[4]: cltol_df = pd.read_csv("Data 148 Project 3 Data/precip_climatology.csv",
    ↪index_col=0)
cltol_df
```

```
[4]:
```

	jan	feb	mar	apr	may \
sahel	1.709537	2.492318	5.714172	16.130378	41.333336
west_afr	8.209585	21.885972	48.073384	81.189124	128.830185
cent_afr	150.822840	135.684500	172.917389	131.626008	66.964828
east_afr	78.856493	77.336063	117.425652	139.091246	94.367546
south_afr	173.127541	159.389011	126.137167	58.982098	20.821174

	jun	jul	aug	sep	oct \
sahel	76.099117	139.665724	164.860457	98.040216	36.330694
west_afr	187.294634	211.482483	235.918114	212.480851	126.953568
cent_afr	38.162833	41.202987	59.097322	81.299915	117.013004
east_afr	57.119627	79.829072	81.796797	64.329445	63.728370
south_afr	13.856495	13.782481	12.926650	15.509177	40.318556

	nov	dec
sahel	6.243360	2.471161
west_afr	40.818007	14.821853
cent_afr	160.153958	166.075571
east_afr	86.672489	84.657042
south_afr	91.751808	159.689238

```
[5]: ssa = pd.read_csv('Data 148 Project 3 Data/gseasSSA.csv', index_col=0)
ssa
```

```
[5]:
```

	country	beg_month	end_month
code			
8	Angola	-2	3
17	Botswana	-1	4
26	Benin	3	8
34	Burundi	-3	2
37	Chad	5	10
39	Congo	-2	3
40	Zaire	-1	4
45	Cameroon	5	10
50	Central African Republic	4	9
56	Djibouti	6	11
65	Eritrea	6	11
67	Ethiopia	4	9
79	Gambia, The	5	10
80	Gabon	-3	2
82	Ghana	4	9
93	Guinea	5	10
110	Ivory Coast	3	8
119	Kenya	3	8
131	Liberia	5	10
134	Lesotho	-2	3
137	Madagascar	-2	3
144	Malawi	-1	4
146	Mali	5	10
151	Mauritania	6	11
158	Mozambique	-1	4
162	Niger	6	11
164	Nigeria	5	10
184	Guinea-Bissau	5	10
192	Rwanda	-3	2
197	South Africa	-1	4
198	Senegal	6	11
201	Sierra Leone	5	10
204	Somalia	4	9
208	Sudan	4	9
221	Togo	4	9
228	Tanzania	1	5
229	Uganda	2	7
233	Burkina Faso	4	9
241	Namibia	0	5
247	Swaziland	-2	3
249	Zambia	-1	4
250	Zimbabwe	-1	4

```
[6]: altogether_monthly = pd.read_csv("altogether.monthly.csv")
altogether_monthly
```

```

[6]:      Jan.a1b.temp.bccr  Feb.a1b.temp.bccr  Mar.a1b.temp.bccr  \
0              NaN              NaN              NaN
1              NaN              NaN              NaN
2              NaN              NaN              NaN
3              NaN              NaN              NaN
4              NaN              NaN              NaN
5              NaN              NaN              NaN
6              NaN              NaN              NaN
7              NaN              NaN              NaN
8              NaN              NaN              NaN
9              NaN              NaN              NaN
10             NaN              NaN              NaN
11             NaN              NaN              NaN
12             NaN              NaN              NaN

      Apr.a1b.temp.bccr  May.a1b.temp.bccr  Jun.a1b.temp.bccr  \
0              NaN              NaN              NaN
1              NaN              NaN              NaN
2              NaN              NaN              NaN
3              NaN              NaN              NaN
4              NaN              NaN              NaN
5              NaN              NaN              NaN
6              NaN              NaN              NaN
7              NaN              NaN              NaN
8              NaN              NaN              NaN
9              NaN              NaN              NaN
10             NaN              NaN              NaN
11             NaN              NaN              NaN
12             NaN              NaN              NaN

      Jul.a1b.temp.bccr  Aug.a1b.temp.bccr  Sep.a1b.temp.bccr  \
0              NaN              NaN              NaN
1              NaN              NaN              NaN
2              NaN              NaN              NaN
3              NaN              NaN              NaN
4              NaN              NaN              NaN
5              NaN              NaN              NaN
6              NaN              NaN              NaN
7              NaN              NaN              NaN
8              NaN              NaN              NaN
9              NaN              NaN              NaN
10             NaN              NaN              NaN
11             NaN              NaN              NaN
12             NaN              NaN              NaN

      Oct.a1b.temp.bccr  ...  Mar.b1.precip.max  Apr.b1.precip.max  \
0              NaN  ...              12.862778              11.479962

```

1	NaN	...	44.675502	13.222318
2	NaN	...	6.665333	10.401119
3	NaN	...	38.289402	12.278062
4	NaN	...	14.238032	18.145620
5	NaN	...	4.570512	9.240330
6	NaN	...	47.478112	28.961655
7	NaN	...	7.998784	8.399136
8	NaN	...	7.037160	9.066743
9	NaN	...	8.856112	9.503279
10	NaN	...	20.264766	15.604641
11	NaN	...	14.642250	9.047707
12	NaN	...	20.603773	10.876095

	May.b1.precip.max	Jun.b1.precip.max	Jul.b1.precip.max	\
0	9.034621	11.768336	7.368951	
1	22.419051	39.861929	20.593185	
2	9.057758	6.087541	8.064493	
3	28.886943	21.268730	9.634491	
4	17.876537	2.740816	7.871675	
5	8.334214	16.529522	5.648388	
6	20.146432	26.568620	9.031788	
7	25.269052	11.645895	6.355432	
8	9.633988	5.249819	21.998737	
9	4.957282	5.891350	14.305034	
10	6.555169	3.632741	10.774268	
11	13.286561	9.355464	11.088268	
12	21.849313	58.308979	28.092052	

	Aug.b1.precip.max	Sep.b1.precip.max	Oct.b1.precip.max	\
0	6.228185	9.601702	14.927341	
1	17.703296	15.972006	14.318148	
2	6.377228	7.550702	6.847648	
3	21.866985	24.033784	23.503554	
4	15.373018	14.643702	14.570823	
5	7.236219	7.529797	9.248344	
6	11.052552	16.423295	26.979200	
7	23.898544	7.073218	5.461134	
8	20.003667	15.293775	13.969640	
9	12.824687	12.048763	3.963186	
10	11.444037	5.275819	6.362274	
11	35.389264	14.274104	13.177250	
12	25.079393	63.072709	39.196689	

	Nov.b1.precip.max	Dec.b1.precip.max
0	15.602040	15.499810
1	26.592497	27.344093
2	6.593600	6.185287

3	25.164238	47.527519
4	23.024443	66.165498
5	12.418194	10.299690
6	32.176674	19.945327
7	3.961043	6.554548
8	8.518846	11.364985
9	6.936252	9.925602
10	11.792821	15.247611
11	14.254357	14.598255
12	31.158238	29.089563

[13 rows x 2016 columns]

0.2 Table 1 Replication Code

0.2.1 We code each individual model before combining them to one table

0.2.2 Model 1

```
[7]: ##Model 1
##dropped 'iccode10' and 'Iccyear10' to avoid dummy variable trap, this was
↳done in replication data too
##dropped NaN values for the specific columns used to avoid dropping too many
↳rows that may be otherwise necessary
##added iccode and Iccyear dummies to replicate what they did in stata
iccode_columns = iccode_columns = climconflict.columns[climconflict.columns.
↳str.startswith('iccode')].drop('iccode10')
Iccyear_columns = climconflict.columns[climconflict.columns.str.
↳startswith('Iccyear')].drop('Iccyear10')
x_columns = ['temp_all', 'temp_all_lag'] + list(iccode_columns) +
↳list(Iccyear_columns)
relevant_columns = x_columns + ['war_prio_new', 'ccode']
climconflict_cleaned = climconflict.dropna(subset = relevant_columns)
X = climconflict_cleaned[x_columns]
X = sm.add_constant(X, has_constant='add')
Y = climconflict_cleaned['war_prio_new']
model_1 = sm.OLS(Y, X).fit(cov_type='cluster', cov_kwds={'groups':
↳climconflict_cleaned['ccode']})
model_1.summary()
```

```
/srv/conda/lib/python3.11/site-packages/statsmodels/base/model.py:1894:
ValueWarning: covariance of constraints does not have full rank. The number of
constraints is 82, but rank is 2
warnings.warn('covariance of constraints does not have full ')
```

[7]:

Dep. Variable:	war_prio_new	R-squared:	0.657
Model:	OLS	Adj. R-squared:	0.622
Method:	Least Squares	F-statistic:	50.44
Date:	Sun, 04 May 2025	Prob (F-statistic):	1.16e-11
Time:	21:05:36	Log-Likelihood:	246.52
No. Observations:	889	AIC:	-327.0
Df Residuals:	806	BIC:	70.54
Df Model:	82		
Covariance Type:	cluster		

	coef	std err	z	P> z	[0.025	0.975]
const	-1.5189	0.925	-1.642	0.101	-3.332	0.294
temp_all	0.0446	0.022	2.058	0.040	0.002	0.087
temp_all_lag	0.0087	0.021	0.415	0.678	-0.032	0.050
iccode1	0.0542	0.066	0.827	0.408	-0.074	0.183
iccode2	0.0730	0.044	1.651	0.099	-0.014	0.160
iccode3	0.0112	0.006	1.799	0.072	-0.001	0.023
iccode4	0.0315	0.019	1.664	0.096	-0.006	0.069
iccode5	0.0298	0.019	1.605	0.108	-0.007	0.066
iccode6	0.0460	0.028	1.645	0.100	-0.009	0.101
iccode7	0.0696	0.042	1.661	0.097	-0.013	0.152
iccode8	0.1163	0.071	1.638	0.101	-0.023	0.255
iccode9	0.1615	0.098	1.653	0.098	-0.030	0.353
iccode11	0.2838	0.108	2.629	0.009	0.072	0.495
iccode12	0.0192	0.086	0.223	0.823	-0.149	0.188
iccode13	0.0523	0.032	1.631	0.103	-0.011	0.115
iccode14	0.0653	0.040	1.630	0.103	-0.013	0.144
iccode15	0.2166	0.132	1.645	0.100	-0.041	0.475
iccode16	0.0815	0.050	1.637	0.102	-0.016	0.179
iccode17	0.1997	0.122	1.642	0.101	-0.039	0.438
iccode18	0.2021	0.122	1.655	0.098	-0.037	0.441
iccode19	0.2801	0.059	4.727	0.000	0.164	0.396
iccode20	0.1304	0.135	0.967	0.333	-0.134	0.395
iccode21	-0.0079	0.144	-0.055	0.956	-0.290	0.274
iccode22	1.6581	0.184	8.988	0.000	1.297	2.020
iccode23	0.2102	0.127	1.659	0.097	-0.038	0.459
iccode24	0.3261	0.199	1.641	0.101	-0.063	0.715
iccode25	0.0785	0.262	0.300	0.764	-0.434	0.591
iccode26	0.4352	0.302	1.439	0.150	-0.157	1.028
iccode27	0.3508	0.055	6.329	0.000	0.242	0.459
iccode28	0.0543	0.031	1.732	0.083	-0.007	0.116
iccode29	1.6224	0.198	8.178	0.000	1.234	2.011
iccode30	1.5431	0.229	6.739	0.000	1.094	1.992
iccode31	1.7135	0.155	11.087	0.000	1.411	2.016
iccode32	0.3610	0.220	1.639	0.101	-0.071	0.793
iccode33	0.3814	0.233	1.637	0.102	-0.075	0.838
iccode34	0.3466	0.211	1.641	0.101	-0.067	0.761
iccode35	1.4904	0.352	4.238	0.000	0.801	2.180
iccode36	0.4394	0.270	1.630	0.103	-0.089	0.968
iccode37	0.8673	0.529	1.641	0.101	-0.169	1.903
iccode38	0.3630	0.223	1.631	0.103	-0.073	0.799
iccode39	0.4226	0.257	1.642	0.101	-0.082	0.927
iccode40	0.3546	0.216	1.644	0.100	-0.068	0.777
iccode41	0.6491	0.045	14.504	0.000	0.561	0.737
Iccyear1	0.0053	0.001	4.451	0.000	0.003	0.008
Iccyear2	-0.0022	0.001	-1.684	0.092	-0.005	0.000
Iccyear3	-0.0017	0.001	-1.732	0.083	-0.004	0.000
Iccyear4	-0.0017	0.001	-1.692	0.091	-0.004	0.000
Iccyear5	0.0008	0.000	1.721	0.085	-0.000	0.002
Iccyear6	-0.0020	0.001	-1.663	0.096	-0.004	0.000
Iccyear7	-0.0012	0.001	-1.718	0.086	-0.003	0.000
Iccyear8	-0.0008	0.000	-1.631	0.103	-0.002	0.000
Iccyear9	-0.0014	0.001	-1.752	0.080	-0.003	0.000

Omnibus:	312.816	Durbin-Watson:	1.509
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3630.287
Skew:	1.258	Prob(JB):	0.00
Kurtosis:	12.575	Cond. No.	7.66e+03

Notes:

- [1] Standard Errors are robust to cluster correlation (cluster)
- [2] The condition number is large, 7.66e+03. This might indicate that there are strong multicollinearity or other numerical problems.

0.2.3 Model 2

```
[8]: ##Model 2
iccode_columns = iccode_columns = climconflict.columns[climconflict.columns.
↳str.startswith('iccode')].drop('iccode10')
Iccyear_columns = climconflict.columns[climconflict.columns.str.
↳startswith('Iccyear')].drop('Iccyear10')
x_columns = ['temp_all', 'temp_all_lag', 'prec_all', 'prec_all_lag'] +
↳list(iccode_columns) + list(Iccyear_columns)
relevant_columns = x_columns + ['war_prio_new', 'ccode']
climconflict_cleaned = climconflict.dropna(subset = relevant_columns)
X = climconflict_cleaned[x_columns]
X = sm.add_constant(X, has_constant='add')
Y = climconflict_cleaned['war_prio_new']
model_2 = sm.OLS(Y, X).fit(cov_type='cluster', cov_kwds={'groups':
↳climconflict_cleaned['ccode']})
model_2.summary()
```

/srv/conda/lib/python3.11/site-packages/statsmodels/base/model.py:1894:
ValueWarning: covariance of constraints does not have full rank. The number of
constraints is 84, but rank is 4

warnings.warn('covariance of constraints does not have full ')

Dep. Variable:	war_prio_new	R-squared:	0.657
Model:	OLS	Adj. R-squared:	0.622
Method:	Least Squares	F-statistic:	30.71
Date:	Sun, 04 May 2025	Prob (F-statistic):	1.03e-11
Time:	21:05:37	Log-Likelihood:	246.69
No. Observations:	889	AIC:	-323.4
Df Residuals:	804	BIC:	83.77
Df Model:	84		
Covariance Type:	cluster		

	coef	std err	z	P> z	[0.025	0.975]
const	-1.5964	0.927	-1.722	0.085	-3.414	0.221
temp_all	0.0428	0.022	1.983	0.047	0.000	0.085
temp_all_lag	0.0132	0.023	0.566	0.571	-0.032	0.059
prec_all	-0.0234	0.052	-0.452	0.652	-0.125	0.078
prec_all_lag	0.0246	0.049	0.506	0.613	-0.071	0.120
iccode1	0.0631	0.084	0.751	0.453	-0.102	0.228
iccode2	0.0787	0.046	1.712	0.087	-0.011	0.169
iccode3	0.0122	0.037	0.334	0.738	-0.060	0.084
iccode4	0.0345	0.022	1.544	0.123	-0.009	0.078
iccode5	0.0312	0.024	1.290	0.197	-0.016	0.079
iccode6	0.0494	0.056	0.883	0.377	-0.060	0.159
iccode7	0.0738	0.060	1.227	0.220	-0.044	0.192
iccode8	0.1214	0.081	1.498	0.134	-0.037	0.280
iccode9	0.1693	0.117	1.449	0.147	-0.060	0.398
iccode11	0.2905	0.155	1.877	0.060	-0.013	0.594
iccode12	0.0234	0.151	0.155	0.877	-0.273	0.320
iccode13	0.0543	0.043	1.276	0.202	-0.029	0.138
iccode14	0.0679	0.047	1.439	0.150	-0.025	0.160
iccode15	0.2261	0.147	1.543	0.123	-0.061	0.513
iccode16	0.0848	0.055	1.546	0.122	-0.023	0.192
iccode17	0.2079	0.146	1.424	0.154	-0.078	0.494
iccode18	0.2115	0.132	1.605	0.109	-0.047	0.470
iccode19	0.2852	0.066	4.321	0.000	0.156	0.415
iccode20	0.1396	0.152	0.917	0.359	-0.159	0.438
iccode21	0.0023	0.157	0.014	0.988	-0.306	0.310
iccode22	1.6732	0.191	8.749	0.000	1.298	2.048
iccode23	0.2212	0.127	1.738	0.082	-0.028	0.471
iccode24	0.3414	0.202	1.693	0.090	-0.054	0.737
iccode25	0.0988	0.267	0.370	0.711	-0.425	0.623
iccode26	0.4590	0.307	1.497	0.134	-0.142	1.060
iccode27	0.3562	0.065	5.492	0.000	0.229	0.483
iccode28	0.0580	0.048	1.214	0.225	-0.036	0.152
iccode29	1.6387	0.199	8.222	0.000	1.248	2.029
iccode30	1.5619	0.231	6.768	0.000	1.110	2.014
iccode31	1.7263	0.157	11.001	0.000	1.419	2.034
iccode32	0.3791	0.223	1.704	0.088	-0.057	0.815
iccode33	0.4018	0.234	1.717	0.086	-0.057	0.861
iccode34	0.3634	0.215	1.691	0.091	-0.058	0.785
iccode35	1.5203	0.353	4.310	0.000	0.829	2.212
iccode36	0.4632	0.272	1.704	0.088	-0.070	0.996
iccode37	0.9121	0.531	1.717	0.086	-0.129	1.953
iccode38	0.3824	0.224	1.708	0.088	-0.056	0.821
iccode39	0.4445	0.259	1.718	0.086	-0.063	0.952
iccode40	0.3721	0.229	1.622	0.105	-0.077	0.822
iccode41	0.6533	0.052	12.472	0.000	0.551	0.756
Iccyear1	0.0049	0.002	2.623	0.009	0.001	0.009
Iccyear2	-0.0024	0.002	-1.585	0.113	-0.005	0.001
Iccyear3	-0.0018	0.001	-1.786	0.074	-0.004	0.000
Iccyear4	-0.0019	0.001	-1.715	0.086	-0.004	0.000
Iccyear5	0.0008	0.001	1.219	0.223	-0.000	0.002
Iccyear6	-0.0021	0.001	-1.741	0.082	-0.004	0.000
Iccyear7	-0.0013	0.001	-1.754	0.079	-0.003	0.000

Omnibus:	313.022	Durbin-Watson:	1.507
Prob(Omnibus):	0.000	Jarque-Bera (JB):	3627.687
Skew:	1.259	Prob(JB):	0.00
Kurtosis:	12.570	Cond. No.	7.97e+03

Notes:

- [1] Standard Errors are robust to cluster correlation (cluster)
- [2] The condition number is large, 7.97e+03. This might indicate that there are strong multicollinearity or other numerical problems.

0.2.4 Model 3

```
[9]: ##Model 3

iccode_columns = iccode_columns = climconflict.columns[climconflict.columns.
↳str.startswith('iccode')].drop('iccode10')
Iccyear_columns = climconflict.columns[climconflict.columns.str.
↳startswith('Iccyear')].drop('Iccyear10')
x_columns = ['temp_all', 'temp_all_lag', 'prec_all', 'prec_all_lag', 'gdp_1',
↳'polity2_lag'] + list(iccode_columns) + list(Iccyear_columns)
relevant_columns = x_columns + ['war_prio_new', 'ccode']
climconflict_cleaned = climconflict.dropna(subset=relevant_columns)
X = climconflict_cleaned[x_columns]
X = sm.add_constant(X, has_constant='add')
Y = climconflict_cleaned['war_prio_new']
model_3 = sm.OLS(Y, X).fit(cov_type='cluster', cov_kwds={'groups':
↳climconflict_cleaned['ccode']})
model_3.summary()
```

/srv/conda/lib/python3.11/site-packages/statsmodels/base/model.py:1894:
ValueWarning: covariance of constraints does not have full rank. The number of
constraints is 86, but rank is 6
warnings.warn('covariance of constraints does not have full ')

```
[9]:
```

Dep. Variable:	war_prio_new	R-squared:	0.621
Model:	OLS	Adj. R-squared:	0.580
Method:	Least Squares	F-statistic:	66.89
Date:	Sun, 04 May 2025	Prob (F-statistic):	2.33e-18
Time:	21:05:38	Log-Likelihood:	222.61
No. Observations:	815	AIC:	-283.2
Df Residuals:	734	BIC:	97.73
Df Model:	80		
Covariance Type:	cluster		

	coef	std err	z	P> z	[0.025	0.975]
const	-1.7855	1.112	-1.606	0.108	-3.965	0.394
temp_all	0.0440	0.024	1.827	0.068	-0.003	0.091
temp_all_lag	0.0183	0.028	0.665	0.506	-0.036	0.072
prec_all	-0.0329	0.056	-0.590	0.555	-0.142	0.076
prec_all_lag	0.0166	0.048	0.342	0.732	-0.078	0.112
gdp_l	2.365e-05	2.42e-05	0.976	0.329	-2.38e-05	7.11e-05
polity2_lag	-0.0003	0.004	-0.094	0.925	-0.008	0.007
iccode1	0.0915	0.109	0.843	0.399	-0.121	0.304
iccode2	0.0913	0.069	1.326	0.185	-0.044	0.226
iccode3	0.0076	0.051	0.148	0.882	-0.093	0.108
iccode4	0.0166	0.031	0.534	0.593	-0.044	0.077
iccode5	0.0336	0.042	0.797	0.426	-0.049	0.116
iccode6	0.0371	0.065	0.567	0.570	-0.091	0.165
iccode7	0.0610	0.078	0.780	0.435	-0.092	0.214
iccode8	0.0985	0.117	0.840	0.401	-0.131	0.328
iccode9	0.1689	0.149	1.136	0.256	-0.123	0.460
iccode11	-0.1776	0.186	-0.956	0.339	-0.541	0.186
iccode12	0.0552	0.175	0.316	0.752	-0.287	0.398
iccode13	0.0542	0.055	0.981	0.326	-0.054	0.163
iccode14	0.0699	0.062	1.122	0.262	-0.052	0.192
iccode15	0.2190	0.186	1.180	0.238	-0.145	0.583
iccode16	0.0990	0.061	1.634	0.102	-0.020	0.218
iccode17	0.0407	0.280	0.145	0.884	-0.508	0.589
iccode18	0.2360	0.168	1.403	0.161	-0.094	0.566
iccode19	0.2949	0.076	3.860	0.000	0.145	0.445
iccode20	0.1289	0.193	0.669	0.503	-0.248	0.506
iccode21	0.0314	0.198	0.158	0.874	-0.357	0.420
iccode22	1.7258	0.220	7.828	0.000	1.294	2.158
iccode23	0.2371	0.159	1.493	0.136	-0.074	0.548
iccode24	0.3947	0.250	1.577	0.115	-0.096	0.885
iccode25	0.1556	0.327	0.475	0.634	-0.486	0.797
iccode26	0.5124	0.373	1.374	0.169	-0.219	1.243
iccode27	0.3496	0.085	4.131	0.000	0.184	0.516
iccode28	-8.84e-16	1.52e-27	-5.84e+11	0.000	-8.84e-16	-8.84e-16
iccode29	1.6859	0.249	6.757	0.000	1.197	2.175
iccode30	1.155e-14	4.47e-26	2.58e+11	0.000	1.16e-14	1.16e-14
iccode31	1.7516	0.204	8.576	0.000	1.351	2.152
iccode32	0.4008	0.283	1.418	0.156	-0.153	0.955
iccode33	0.3897	0.278	1.404	0.160	-0.154	0.934
iccode34	0.4107	0.276	1.486	0.137	-0.131	0.952
iccode35	1.4235	0.448	3.177	0.001	0.545	2.302
iccode36	0.4107	0.310	1.325	0.185	-0.197	1.018
iccode37	-3.512e-15	1.89e-27	-1.86e+12	0.000	-3.51e-15	-3.51e-15
iccode38	0.4103	0.255	1.607	0.108	-0.090	0.911
iccode39	0.4143	0.331	1.251	0.211	-0.235	1.063
iccode40	0.4214	0.283	1.490	0.136	-0.133	0.976
iccode41	0.6547	0.058	11.305	0.000	0.541	0.768
Iccyear1	0.0052	0.004	1.323	0.186	-0.002	0.013
Iccyear2	-0.0028	0.003	13 -1.018	0.309	-0.008	0.003
Iccyear3	-0.0020	0.004	-0.514	0.607	-0.010	0.006
Iccyear4	-0.0018	0.002	-1.005	0.315	-0.005	0.002
Iccyear5	0.0011	0.003	0.350	0.726	-0.005	0.007

Omnibus:	273.447	Durbin-Watson:	1.517
Prob(Omnibus):	0.000	Jarque-Bera (JB):	2659.153
Skew:	1.229	Prob(JB):	0.00
Kurtosis:	11.501	Cond. No.	9.74e+20

Notes:

[1] Standard Errors are robust to cluster correlation (cluster)

[2] The smallest eigenvalue is 6.62e-33. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

0.2.5 Combined results of all models

```
[10]: from statsmodels.iolib.summary2 import summary_col

results_table = summary_col([model_1, model_2, model_3], stars=True,
                             model_names=['Model 1', 'Model 2', 'Model 3'])
print(results_table)
##standard error under coefficient and in parenthesis ()
```

	Model 1	Model 2	Model 3
const	-1.5189 (0.9250)	-1.5964* (0.9272)	-1.7855 (1.1121)
temp_all	0.0446** (0.0217)	0.0428** (0.0216)	0.0440* (0.0241)
temp_all_lag	0.0087 (0.0209)	0.0132 (0.0233)	0.0183 (0.0275)
iccode1	0.0542 (0.0655)	0.0631 (0.0840)	0.0915 (0.1085)
iccode2	0.0730* (0.0442)	0.0787* (0.0460)	0.0913 (0.0688)
iccode3	0.0112* (0.0062)	0.0122 (0.0367)	0.0076 (0.0511)
iccode4	0.0315* (0.0189)	0.0345 (0.0224)	0.0166 (0.0310)
iccode5	0.0298 (0.0186)	0.0312 (0.0242)	0.0336 (0.0422)
iccode6	0.0460 (0.0280)	0.0494 (0.0559)	0.0371 (0.0654)
iccode7	0.0696* (0.0419)	0.0738 (0.0602)	0.0610 (0.0782)
iccode8	0.1163 (0.0710)	0.1214 (0.0811)	0.0985 (0.1173)
iccode9	0.1615* (0.0977)	0.1693 (0.1168)	0.1689 (0.1487)
iccode11	0.2838***	0.2905*	-0.1776

	(0.1079)	(0.1547)	(0.1857)
iccode12	0.0192	0.0234	0.0552
	(0.0859)	(0.1512)	(0.1748)
iccode13	0.0523	0.0543	0.0542
	(0.0320)	(0.0425)	(0.0553)
iccode14	0.0653	0.0679	0.0699
	(0.0401)	(0.0472)	(0.0623)
iccode15	0.2166*	0.2261	0.2190
	(0.1317)	(0.1465)	(0.1856)
iccode16	0.0815	0.0848	0.0990
	(0.0498)	(0.0548)	(0.0606)
iccode17	0.1997	0.2079	0.0407
	(0.1216)	(0.1459)	(0.2798)
iccode18	0.2021*	0.2115	0.2360
	(0.1221)	(0.1318)	(0.1682)
iccode19	0.2801***	0.2852***	0.2949***
	(0.0592)	(0.0660)	(0.0764)
iccode20	0.1304	0.1396	0.1289
	(0.1348)	(0.1523)	(0.1925)
iccode21	-0.0079	0.0023	0.0314
	(0.1439)	(0.1571)	(0.1983)
iccode22	1.6581***	1.6732***	1.7258***
	(0.1845)	(0.1912)	(0.2205)
iccode23	0.2102*	0.2212*	0.2371
	(0.1267)	(0.1273)	(0.1589)
iccode24	0.3261	0.3414*	0.3947
	(0.1987)	(0.2016)	(0.2503)
iccode25	0.0785	0.0988	0.1556
	(0.2617)	(0.2672)	(0.3272)
iccode26	0.4352	0.4590	0.5124
	(0.3023)	(0.3067)	(0.3729)
iccode27	0.3508***	0.3562***	0.3496***
	(0.0554)	(0.0649)	(0.0846)
iccode28	0.0543*	0.0580	-0.0000***
	(0.0313)	(0.0478)	(0.0000)
iccode29	1.6224***	1.6387***	1.6859***
	(0.1984)	(0.1993)	(0.2495)
iccode30	1.5431***	1.5619***	0.0000***
	(0.2290)	(0.2308)	(0.0000)
iccode31	1.7135***	1.7263***	1.7516***
	(0.1545)	(0.1569)	(0.2042)
iccode32	0.3610	0.3791*	0.4008
	(0.2203)	(0.2225)	(0.2826)
iccode33	0.3814	0.4018*	0.3897
	(0.2330)	(0.2340)	(0.2775)
iccode34	0.3466	0.3634*	0.4107
	(0.2112)	(0.2149)	(0.2763)
iccode35	1.4904***	1.5203***	1.4235***

	(0.3517)	(0.3528)	(0.4481)
iccode36	0.4394	0.4632*	0.4107
	(0.2696)	(0.2719)	(0.3100)
iccode37	0.8673	0.9121*	-0.0000***
	(0.5285)	(0.5313)	(0.0000)
iccode38	0.3630	0.3824*	0.4103
	(0.2225)	(0.2239)	(0.2553)
iccode39	0.4226	0.4445*	0.4143
	(0.2573)	(0.2588)	(0.3311)
iccode40	0.3546	0.3721	0.4214
	(0.2157)	(0.2293)	(0.2828)
iccode41	0.6491***	0.6533***	0.6547***
	(0.0448)	(0.0524)	(0.0579)
Iccyear1	0.0053***	0.0049***	0.0052
	(0.0012)	(0.0019)	(0.0039)
Iccyear2	-0.0022*	-0.0024	-0.0028
	(0.0013)	(0.0015)	(0.0028)
Iccyear3	-0.0017*	-0.0018*	-0.0020
	(0.0010)	(0.0010)	(0.0040)
Iccyear4	-0.0017*	-0.0019*	-0.0018
	(0.0010)	(0.0011)	(0.0018)
Iccyear5	0.0008*	0.0008	0.0011
	(0.0004)	(0.0007)	(0.0031)
Iccyear6	-0.0020*	-0.0021*	-0.0026
	(0.0012)	(0.0012)	(0.0016)
Iccyear7	-0.0012*	-0.0013*	-0.0007
	(0.0007)	(0.0007)	(0.0028)
Iccyear8	-0.0008	-0.0008	-0.0002
	(0.0005)	(0.0007)	(0.0022)
Iccyear9	-0.0014*	-0.0015	-0.0017
	(0.0008)	(0.0009)	(0.0023)
Iccyear11	-0.0022***	-0.0022*	0.0652***
	(0.0006)	(0.0013)	(0.0037)
Iccyear12	0.0143***	0.0143***	0.0150***
	(0.0009)	(0.0008)	(0.0023)
Iccyear13	-0.0001	-0.0001	0.0002
	(0.0001)	(0.0003)	(0.0018)
Iccyear14	0.0006*	0.0007	0.0015
	(0.0003)	(0.0007)	(0.0015)
Iccyear15	-0.0010*	-0.0010	-0.0003
	(0.0006)	(0.0010)	(0.0018)
Iccyear16	-0.0002	-0.0002	-0.0002
	(0.0002)	(0.0006)	(0.0008)
Iccyear17	-0.0018*	-0.0018	-0.0010
	(0.0010)	(0.0011)	(0.0023)
Iccyear18	-0.0013*	-0.0013*	-0.0008
	(0.0007)	(0.0007)	(0.0034)
Iccyear19	-0.0083***	-0.0084***	-0.0088***

	(0.0009)	(0.0009)	(0.0012)
Iccyear20	0.0119***	0.0119***	0.0135***
	(0.0010)	(0.0010)	(0.0027)
Iccyear21	0.0303***	0.0303***	0.0313***
	(0.0007)	(0.0007)	(0.0029)
Iccyear22	-0.0680***	-0.0681***	-0.0688***
	(0.0008)	(0.0009)	(0.0014)
Iccyear23	-0.0011*	-0.0011*	-0.0012
	(0.0006)	(0.0006)	(0.0013)
Iccyear24	-0.0011*	-0.0011*	-0.0014
	(0.0007)	(0.0007)	(0.0020)
Iccyear25	0.0384***	0.0385***	0.0386***
	(0.0006)	(0.0006)	(0.0018)
Iccyear26	0.0169***	0.0169***	0.0171***
	(0.0007)	(0.0007)	(0.0011)
Iccyear27	-0.0066***	-0.0066***	-0.0060***
	(0.0005)	(0.0005)	(0.0021)
Iccyear28	-0.0021*	-0.0022*	-0.0000***
	(0.0011)	(0.0011)	(0.0000)
Iccyear29	-0.0638***	-0.0638***	-0.0639***
	(0.0009)	(0.0009)	(0.0026)
Iccyear30	-0.0250***	-0.0251***	0.0000***
	(0.0014)	(0.0014)	(0.0000)
Iccyear31	-0.0688***	-0.0689***	-0.0685***
	(0.0006)	(0.0007)	(0.0037)
Iccyear32	-0.0013	-0.0014*	-0.0003
	(0.0008)	(0.0008)	(0.0037)
Iccyear33	-0.0006	-0.0007	-0.0006
	(0.0004)	(0.0006)	(0.0015)
Iccyear34	-0.0013*	-0.0014*	-0.0013
	(0.0008)	(0.0008)	(0.0039)
Iccyear35	-0.0478***	-0.0478***	-0.0478***
	(0.0002)	(0.0003)	(0.0014)
Iccyear36	-0.0006	-0.0007	-0.0006
	(0.0005)	(0.0007)	(0.0017)
Iccyear37	-0.0000	-0.0001	0.0000***
	(0.0000)	(0.0004)	(0.0000)
Iccyear38	-0.0011	-0.0012	-0.0071
	(0.0007)	(0.0008)	(0.0061)
Iccyear39	-0.0003*	-0.0004	-0.0038
	(0.0002)	(0.0004)	(0.0036)
Iccyear40	-0.0015*	-0.0016	-0.0012
	(0.0009)	(0.0010)	(0.0040)
Iccyear41	0.0166***	0.0165***	0.0162***
	(0.0008)	(0.0009)	(0.0011)
prec_all		-0.0234	-0.0329
		(0.0517)	(0.0557)
prec_all_lag		0.0246	0.0166

		(0.0486)	(0.0484)
gdp_l			0.0000
			(0.0000)
polity2_lag			-0.0003
			(0.0037)
R-squared	0.6572	0.6573	0.6212
R-squared Adj.	0.6223	0.6215	0.5799

=====

Standard errors in parentheses.
 * p<.1, ** p<.05, ***p<.01

0.3 Preliminary code required to perform Figure 1, Table 2, and Figure 2

0.3.1 Here we use altogether monthly to create a new, usable, matrix like the original dataset

```
[11]: ##The main purpose of this code is to take the dataframe "altogether.monthly,"
      ↳that was not originally a dataframe, and to turn it into an array to use for
      ↳later data replication
      ##
      df_altogether = altogether_monthly.copy()
      df_altogether['region'] = range(df_altogether.shape[0])
      df_melted = df_altogether.melt(id_vars='region', var_name='col',
      ↳value_name='value')
      col_split = df_melted['col'].str.extract(r'(?P<month>\w+)\.(?P<scenario>[^\.]+\.)\.(?P<variable>[^\.]+\.)\.(?P<model>.+)'
      ↳to_xarray()['value'])
      df_melted = pd.concat([df_melted[['region', 'value']], col_split], axis=1)
      df_melted = df_melted.dropna()
      ds = df_melted.set_index(['region', 'month', 'scenario', 'variable', 'model']).
      ↳to_xarray()['value']
      ds.shape
      print(ds.dims)
```

('region', 'month', 'scenario', 'variable', 'model')

```
[12]: ##using a properly formatted "altogether.monthly" we now have access to the
      ↳data necessary to perform projections and data replication
      ##following the r code replication we specify regions and create code for temp
      ↳projection, precipitation projection, create a new region called "ALLSA" as
      ↳named in the r code
      ##and create a 6 region data set with 3 scenarios (a1b, a2, and b1), variables,
      ↳and models
      regions_of_interest = ds.region[3:8]

      # Mean temp over months for each region, scenario, and model
      temp_proj = ds.sel(region=regions_of_interest, variable='temp').
      ↳mean(dim='month')
```

```

# shape: (region=5, scenario=3, model=20)

#precipitation projection
cltol_da = xr.DataArray(
    cltol_df.values,
    coords={'region': regions_of_interest, 'month': ds.month},
    dims=['region', 'month']
)

# Get projected % change in precip
precip_proj = ds.sel(region=regions_of_interest, variable='precip') / 100 + 1

# Apply to baseline climatology
precip_weighted = cltol_da * precip_proj

# Sum weighted, divide by baseline total, subtract 1, scale to percent
precip_pct_change = ((precip_weighted.sum(dim='month') / cltol_da.
    ↪sum(dim='month')) - 1) * 100
# shape: (region=5, scenario=3, model=20)

# Create final clim xarray with region, scenario, variable, model
clim = xr.concat([temp_proj, precip_pct_change], dim='variable')
clim = clim.transpose('region', 'scenario', 'variable', 'model')

# Compute unweighted average across the 5 regions (dim 0)
ssa_avg = clim.mean(dim='region')

# Add a new label for the synthetic 'AllSSA' region
ssa_avg = ssa_avg.expand_dims({'region': ['AllSSA']})

# Concatenate with the original 5 regions to form a 6-region dataset
clim_full = xr.concat([clim, ssa_avg], dim='region')

```

```

[32]: ##we utilized this code to de-bug and ensure our data was properly coded
##we wanted to see its shape and determine if it matches with the replication
    ↪data

print(clim_full.shape)
print(clim_full)
print(clim_full.sel(region='AllSSA', scenario='a1b'))

```

```

(6, 3, 2, 28)
[[[ 18.          0.98496847  1.49172855 ...  1.16420447
      0.96025262  0.8123312 ]
 [ 18.          -5.77391323  6.70574672 ... -4.09938718
      1.63383421  4.10865693]]

```

```

[[ 15.          0.85886982   1.26720986 ...   0.93341509
   0.81761876   0.67334579]
 [ 15.          -3.58288833   8.91783032 ...  -4.56376998
   0.92502798   3.49872002]]

[[ 18.          0.77869978   1.24965413 ...   0.89737561
   0.69262425   0.96117384]
 [ 18.          -5.60512989   7.83763757 ...  -7.6238368
   0.28601501   3.57426189]]]

[[[ 18.          0.81327003   1.26181362 ...   0.84863375
   0.78115406   0.64951147]
 [ 18.          -6.15790107   6.60011473 ...   1.65189305
  -1.13241515   9.87335251]]]

[[ 15.          0.75453537   1.13006872 ...   0.56719146
   0.71089681   0.51769933]
 [ 15.          -5.15483331   6.56671503 ...   4.01759762
  -0.88003428   5.50105114]]]

[[ 18.          0.64554993   1.07670493 ...   0.65197319
   0.56348798   0.74510312]
 [ 18.          -5.13426214   8.43807333 ...   3.1626616
  -2.23490254  13.32451601]]]

[[[ 18.          0.83140992   1.23261024 ...   0.97384475
   0.74611327   0.59795903]
 [ 18.          -1.29275628   3.2995193 ...  -0.38406463
   1.0478398    0.99981878]]]

[[ 15.          0.77027731   1.08541527 ...   0.7985323
   0.68439014   0.43864838]
 [ 15.          -0.97451373   3.8304659 ...  -0.17573768
   0.98003455   1.16719541]]]

[[ 18.          0.65676117   0.9996889 ...   0.77576497
   0.55366766   0.6785067 ]
 [ 18.          -1.1288258    3.57226013 ...  -0.56386382
   1.17515422   2.79691225]]]

[[[ 18.          0.7876226    1.21383881 ...   0.91037237
   0.78372384   0.70562725]
 [ 18.          -0.1157447    8.81371314 ...  13.15694633
   2.62021231   3.94861874]]]

```

```

[[ 15.          0.77317416   1.04405107 ...   0.78889499
   0.70395482   0.55694617]
 [ 15.         -1.20032744  10.11036582 ...  10.43027721
   2.92345747  -0.47860484]]

[[ 18.          0.63252378   0.99997796 ...   0.71556783
   0.60109394   0.74422295]
 [ 18.         -0.28796436   9.67162285 ...  11.02482736
   1.37873935   2.67444871]]]

[[[ 18.          0.92131013   1.30754682 ...   1.03930091
   0.82756995   0.7250631 ]
 [ 18.         -11.71108504   1.10778458 ...  -9.39307974
  -5.1575431    2.38264367]]]

[[ 15.          0.88109823   1.18003503 ...   0.88417388
   0.77576068   0.53771776]
 [ 15.         -8.49817381   0.80543437 ...  -3.75096115
  -4.34857831   0.87292064]]]

[[ 18.          0.73015468   1.10524846 ...   0.87014595
   0.62513995   0.74036901]
 [ 18.         -8.45558696  -0.65080333 ...  -8.06680096
  -4.01115219   0.13349117]]]

[[[ 18.          0.86771623   1.30150761 ...   0.98727125
   0.81976275   0.69809841]
 [ 18.         -5.01028007   5.30537569 ...   0.18646157
  -0.19761438   4.26261813]]]

[[ 15.          0.80759098   1.14135599 ...   0.79444154
   0.73852424   0.54487149]
 [ 15.         -3.88214732   6.04616229 ...   1.1914812
  -0.08001852   2.11225647]]]

[[ 18.          0.68873786   1.08625488 ...   0.78216551
   0.60720276   0.77387512]
 [ 18.         -4.12235383   5.77375811 ...  -0.41340252
  -0.68122923   4.50072601]]]]

```

AttributeError

Traceback (most recent call last)

Cell In[32], line 6

```

4 print(clim_full.shape)
5 print(clim_full)

```

```
----> 6 print(clim_full.sel(region='AllSSA', scenario='a1b'))

AttributeError: 'numpy.ndarray' object has no attribute 'sel'
```

0.3.2 We save the new 'clim_full' dataset to use later

```
[14]: ##We save our new data set as a npy to load in later for other data replication
np.save('clim_full.npy', clim_full)
```

0.4 Bootstrapping

0.4.1 The paper utilizes bootstrapping of necessary variables in their creation of tables

```
[15]: ##The most essential part, and most difficult is the bootstrapping
##Here we are bootstrapping 9 different combinations of variables necessary for
our data replication, this was pulled from the replication r code and
translated to python
##we run ols regression with these unique combinations that will be used later
##we run 1000 bootstrap iterations instead of 10,000 for time sake

hold = []

# Define the 9 regression specifications based on column index positions
# Adjust indices by -1 for zero-based indexing in Python
hold.append(climconflict.iloc[:, [105, 41, 44] + list(range(109, 150)) +
↳list(range(177, 218))])
hold.append(climconflict.iloc[:, [105, 41, 44, 53, 56] + list(range(109, 150))
↳+ list(range(177, 218))])
hold.append(climconflict.iloc[:, [105, 41, 44, 9, 227] + list(range(109, 150))
↳+ [176]])
hold.append(climconflict.iloc[:, [105, 41, 44, 53, 56, 9, 227] +
↳list(range(109, 150)) + [176]])
hold.append(climconflict.iloc[:, [105, 41, 44, 53, 56] + list(range(109, 150))
↳+ [176]])
hold.append(climconflict.iloc[:, [105, 47, 50, 59, 62] + list(range(109, 150))
↳+ list(range(177, 218))])
hold.append(climconflict.iloc[:, [105] + list(range(218, 223)) +
↳list(range(109, 150)) + list(range(177, 218))])
hold.append(climconflict.iloc[:, [105, 41, 44, 10, 11] + list(range(109, 150))
↳+ list(range(177, 218))])
hold.append(climconflict.iloc[:, [105, 69, 76, 70, 77] + list(range(109, 150))
↳+ list(range(177, 218))])

bootstrap_results = []
```

```

n_bootstrap = 1000 # number of bootstrap iterations
for j, df in enumerate(hold, start=1):
    df_clean = df.dropna()
    X_cols = df_clean.columns[1:] # exclude dependent variable (assumed to be
    ↪in column 0)
    y_col = df_clean.columns[0]

    coefs = []
    print(f"Running bootstrap for model spec {j} with {df_clean.shape[0]}
    ↪observations...")

    for _ in tqdm(range(n_bootstrap)):
        sample = df_clean.sample(n=len(df_clean), replace=True)
        X = sm.add_constant(sample[X_cols])
        y = sample[y_col]
        model = sm.OLS(y, X).fit()
        coefs.append(model.params.values[1:7]) # store coefficients 1 to 6

    coefs = np.vstack(coefs)
    bootstrap_results.append(coefs)

# Save to file
np.save(f'bootstrap_{j}.npy', coefs)

```

Running bootstrap for model spec 1 with 889 observations...

100%| | 1000/1000 [07:16<00:00, 2.29it/s]

Running bootstrap for model spec 2 with 889 observations...

100%| | 1000/1000 [05:25<00:00, 3.07it/s]

Running bootstrap for model spec 3 with 815 observations...

100%| | 1000/1000 [02:52<00:00, 5.78it/s]

Running bootstrap for model spec 4 with 815 observations...

100%| | 1000/1000 [11:33<00:00, 1.44it/s]

Running bootstrap for model spec 5 with 889 observations...

100%| | 1000/1000 [01:56<00:00, 8.61it/s]

Running bootstrap for model spec 6 with 889 observations...

100%| | 1000/1000 [13:57<00:00, 1.19it/s]

Running bootstrap for model spec 7 with 814 observations...

100%| | 1000/1000 [06:33<00:00, 2.54it/s]

Running bootstrap for model spec 8 with 889 observations...

100%| | 1000/1000 [05:21<00:00, 3.11it/s]

Running bootstrap for model spec 9 with 809 observations...

100%| | 1000/1000 [05:05<00:00, 3.28it/s]

```
[17]: ##we ran this code to ensure all new data was saved

import os

# Check for bootstrap files
for i in [1, 2, 4]:
    path = f'bootstrap_{i}.npz'
    print(f'{path}: ', os.path.exists(path))

# Check for climate projections file
print('clim_full.npz: ', os.path.exists('clim_full.npz'))
```

bootstrap_1.npz: True

bootstrap_2.npz: True

bootstrap_4.npz: True

clim_full.npz: True

0.5 Table 2

```
[33]: ##Table 2

# List of bootstrap file paths, loading in bootstrap files
bootstrap_files = [f'./bootstrap_{i}.npz' for i in range(1, 5)]

# Load the climate projections
clim_full = np.load("clim_full.npz")

# Example climatology data
clt01_df = pd.read_csv("Data 148 Project 3 Data/precip_climatology.csv",
    ↪index_col=0)

# Initialize list to hold coefficients from each bootstrap sample
hold = []

# Load bootstrap coefficient files, table 2 utilizes these 3 specific bootstraps
bootstrap_files = ["bootstrap_1.npz", "bootstrap_2.npz", "bootstrap_4.npz"]
for file in bootstrap_files:
    coeff = np.load(file)
    hold.append(coeff)

# Scenario labels
scenarios = ['a1b', 'a2', 'b1']
```



```

variables = ['temp', 'precip']
region_idx = 5 # refers to the ALLSA region created

# Initialize an empty projections matrix to store results
projections = np.zeros((6, 5)) # 6 rows, 5 columns as per Table 2 structure
projections_rows = ["A1btemp", "A1btemp_prec", "A2temp", "A2temp_prec",
    ↪ "B1temp", "B1temp_prec"]

# Base conflict incidence
base = np.mean(climconflict['war_prio_new'][climconflict['year_actual'] < 2003])

# Loop over each bootstrap iteration and calculate projections
for i in range(2): # i corresponds to the bootstrap index (1 or 2 in the
    ↪ original R code)
    coeff = hold[i]
    k = region_idx # For "AllSSA"

    # Loop over scenarios (A1b, A2, B1)
    for x in range(3): # 3 scenarios
        projt = clim_full[k, x, 0, :] # Temperature projections (0 index for
        ↪ 'temp')
        projp = clim_full[k, x, 1, :] * np.mean(climconflict.iloc[:,53],
        ↪ axis=0) / 100 # Precip projections

        # Remove NaNs from projections (in case of missing values)
        projt = projt[~np.isnan(projt)]
        projp = projp[~np.isnan(projp)]

        all_projections = []
        if i == 0: # When i is 0, set precip coefficient terms to 0 (for
        ↪ temperature-only model)
            coeff[:, 2:6] = 0
        if i == 1: # When i is 1, set temperature coefficient terms to 0 (for
        ↪ precipitation-only model)
            coeff[:, 4:6] = 0

        # Calculate projections using the coefficients and projections
        for j in range(len(projt)):
            all_projections.append(projt[j] * (coeff[:, 0] + coeff[:, 1]) +
        ↪ projp[j] * (coeff[:, 2] + coeff[:, 3]))

        # Sort the projections for statistical analysis
        all_projections = np.sort(all_projections) * 100 # Scaling by 100 as
        ↪ per the original code

        all_projections[all_projections == -100] = np.nan

```

```

all_projections_cleaned = all_projections[~np.isnan(all_projections)]

# Compute the summary statistics
all10 = all_projections_cleaned
all10 = all10[~np.isnan(all10)]
median_all10 = np.median(all10)
median_normalized = median_all10 / base
percentile_5th = np.percentile(all10, 5) / base # 5th percentile
↳normalized
percentile_95th = np.percentile(all10, 95) / base # 95th percentile
↳normalized
percent_neg = np.sum(all10 < 0) / len(all10) * 100

# Update projections matrix
projections[i * 3 + x, :] = [
    median_all10,
    median_normalized,
    percentile_5th,
    percentile_95th,
    percent_neg
]

projections_df = pd.DataFrame(projections, columns=["median", "med_inc",
↳"5th_inc", "95th_inc", "% obs<0"])
projections_df.index = projections_rows

print(projections_df)

```

	median	med_inc	5th_inc	95th_inc	% obs<0
A1btemp	5.687937	51.597717	2.589266	143.094563	4.100000
A1btemp_prec	4.975757	45.137223	2.236228	127.287866	4.100000
A2temp	4.585053	41.592983	2.179887	123.276147	4.100000
A2temp_prec	5.725719	51.940450	-2.448373	159.264575	5.800000
B1temp	5.044967	45.765056	-10.030348	153.432061	8.900000
B1temp_prec	4.612152	41.838811	-3.108922	139.577592	6.415385

0.6 Figure 2

```

[29]: # Load bootstrap coefficient files
      ##figure 2 utilizes nearly all bootstrap iterations

bootstrap_files = [f'./bootstrap_{i}.npy' for i in range(1, 9)]

hold = []

bootstrap_files = [

```

```

    "bootstrap_1.npy", "bootstrap_2.npy", "bootstrap_6.npy", "bootstrap_7.npy",
    "bootstrap_8.npy", "bootstrap_9.npy", "bootstrap_4.npy", "bootstrap_4.npy"
]

for file in bootstrap_files:
    coeff = np.load(file)
    hold.append(coeff)

# --- Income and political change projections ---
ystart = climconflict.loc[climconflict['year_actual'] == 1981, climconflict.
    ↪columns[9]]
yend = climconflict.loc[climconflict['year_actual'] == 2002, climconflict.
    ↪columns[9]]

ygro = np.exp(np.log(yend.values / ystart.values) / 21)
yend_median = np.nanmedian(yend.values)
ygro_median = np.nanmedian(ygro)
incchg = np.array([
    yend_median * (1.02 ** 27),
    yend_median * (ygro_median ** 27)
]) - yend_median

polstart = climconflict.loc[climconflict['year_actual'] == 1981, climconflict.
    ↪columns[227]]
polend = climconflict.loc[climconflict['year_actual'] == 2002, climconflict.
    ↪columns[227]]
polchg_raw = polend.values - polstart.values
pol2030 = polend.values + np.nanmedian(polchg_raw)
pol2030 = np.where(pol2030 > 10, 10, pol2030)
polchg = np.nanmean(pol2030 - polend.values)

# --- Plotting ---
fig, ax = plt.subplots(figsize=(8, 8))
ax.set_xlim(-5, 15)
ax.set_ylim(0, len(hold) + 1)
ax.set_xlabel("percentage point change in civil war", fontsize=11)
ax.set_yticks([])
ax.tick_params(axis='y', left=False, labelleft=False)
ax.axvline(x=0, linestyle='dotted')

nms = [
    "Temperature only", "Temperature and precipitation",
    "Temperature and precipitation,", "Temperature and precipitation,",
    "Temperature (CRU) and precipitation (GPCP)",
    "Temperature and precipitation (NCC)",
    "Temperature and precipitation, linear",
    "Temperature and precipitation,"

```

```

]
nms2 = [
    "(Table 1, Model 1)", "(Table 1, Model 2)",
    " first differences (Table S2, Model 3)",
    "deviations from trend (Table S2, Model 5)",
    "(Table S3, Model 1)", "(Table S3, Model 3)",
    "extrapolation of 1981-02 income and", "optimistic scenario for income and"
]
nms3 = [" for _ in range(6)] + [
    "democracy trends (Table 1, Model 3)",
    "democracy trends (Table 1, Model 3)"
]

min_len = min(len(projt), len(projp))
projt = projt[:min_len]
projp = projp[:min_len]
valid_idx = ~np.isnan(projt) & ~np.isnan(projp)
projt = projt[valid_idx]
projp = projp[valid_idx]

k = 5 # AllSSA
projt = np.maximum(projt, 0) # Ensure temperature projections are non-negative
projp = np.maximum(projp, 0)
projt = projt / np.max(projt) # Normalize to the max value
projp = projp / np.max(projp)
projp[projp == -9.84650664e-01] = np.nan # Replace invalid values with NaN
projp = projp[~np.isnan(projp)]
projt = clim_full[k, 1, 0, :].copy() # temp projections under A1B
projp_raw = clim_full[k, 1, 1, :].copy()
continental_precip_mean = np.nanmean(climconflict.iloc[:, 56])
projp = projp_raw * continental_precip_mean / 100 #adjusting for precipitaiton
projp = projp[~np.isnan(projp)]
projt = projt[~np.isnan(projt)]

where = np.arange(len(hold), 0, -1) - 0.1
where[where < 2.75] -= 0.7

for i in range(len(hold)):
    coef = hold[i].copy()
    if i == 0:
        coef[:, 3:6] = 0 #like in table 2, we set no necessary coefficients to
        ↪ zero
    elif i in range(1, 6):
        coef[:, 4:6] = 0

    inc = 0
    if i == 7:

```

```

        inc = incchg[1]
    if i == 8:
        inc = incchg[0]

    all_proj = []
    for j in range(len(projt)):
        pred = (projt[j] * (coef[:, 0] + coef[:, 1]) +
                projp[j] * (coef[:, 2] + coef[:, 3]) +
                inc * coef[:, 4] +
                polchg * coef[:, 5])
        all_proj.extend(pred)

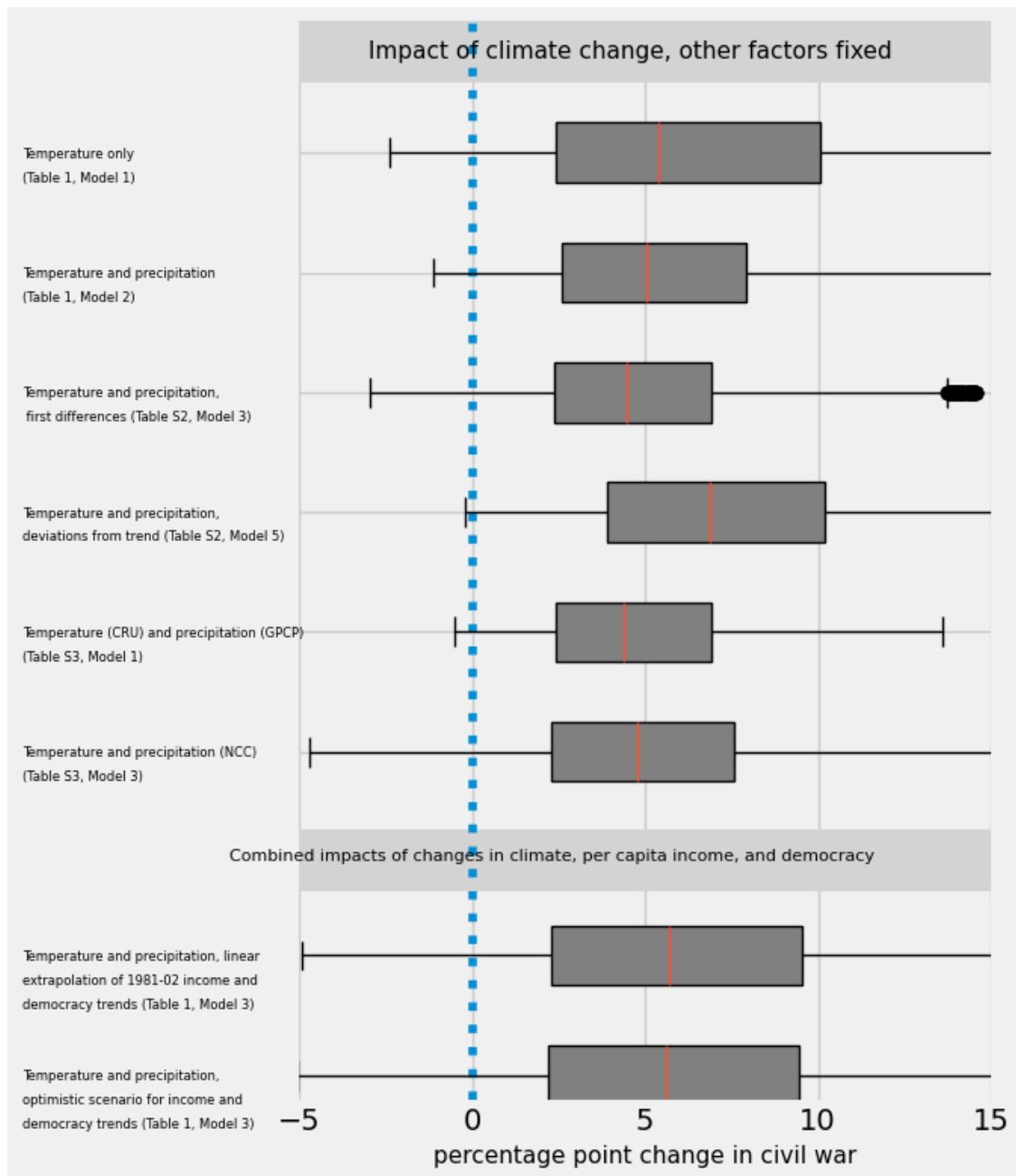
    all_proj = np.sort(np.array(all_proj)) * 100
    n = len(all_proj)
    all10 = all_proj[int(n * 0.05): int(n * 0.95)]
    ax.boxplot(all10, positions=[where[i]], vert=False, widths=0.5,
        patch_artist=True,
        boxprops=dict(facecolor='grey'))
    ax.text(-13, where[i], nms[i], fontsize=6, ha='left', va='center')
    ax.text(-13, where[i] - 0.2, nms2[i], fontsize=6, ha='left', va='center')
    ax.text(-13, where[i] - 0.4, nms3[i], fontsize=6, ha='left', va='center')

# Add section titles
ax.add_patch(plt.Rectangle((-30, len(hold) + 0.5), 50, 1, color='lightgrey'))
ax.text(-3, len(hold) + 0.7, "Impact of climate change, other factors fixed",
    fontsize=11, ha='left')

ax.add_patch(plt.Rectangle((-30, 1.75), 50, 0.5, color='lightgrey'))
ax.text(-7, 2, "Combined impacts of changes in climate, per capita income, and
    democracy", fontsize=8, ha='left')

plt.box(False)
plt.tight_layout()
plt.show()

```



0.7 Figure 1

0.7.1 We divide this visualization replication into two parts, left and right, according to the paper

Figure 1 left side

[30]: `##Figure 1`
`# Figure 1 Left Side`

```

# Region labels and colors
clim_full = np.load('clim_full.npy', allow_pickle=True)
bootstrap = np.load('bootstrap_1.npy')
region_labels = ["Sahel", "West Africa", "Central Africa", "Eastern Africa",
    ↪ "Southern Africa", "Sub-Saharan Africa"]
colors = ['red', 'orange', 'green', 'blue', 'grey', 'white']
scenario_idx = 0 # A1B scenario

# Load precipitation and temperature projections
precip_data = clim_full[:, scenario_idx, 1, :] # shape (6, 28)
temp_data = clim_full[:, scenario_idx, 0, :] # shape (6, 28)

temp_data_cleaned = temp_data
# Remove NaNs
temp_data_cleaned = np.where(np.isnan(temp_data_cleaned), np.nan,
    ↪ temp_data_cleaned)

fig, axs = plt.subplots(2, 2, figsize=(16, 10), gridspec_kw={'width_ratios':
    ↪ [2, 2]})

# -----
# LEFT-TOP: Precipitation Change (%)
# -----
axs[0, 0].set_title("Projected Precipitation Change (%), up to 2030 (A1B)")
axs[0, 0].axvline(0, color='black', linestyle='--')
axs[0, 0].set_xlim(-20, 40)
for i in range(6):
    data = precip_data[i, :]
    sorted_data = np.sort(data)
    axs[0, 0].boxplot(
        sorted_data,
        positions=[i],
        widths=0.6,
        vert=False,
        patch_artist=True,
        boxprops=dict(facecolor=colors[i], color='black'),
        whiskerprops=dict(color='black'),
        capprops=dict(color='black'),
        medianprops=dict(color='black')
    )
axs[0, 0].set_yticks(range(6))
axs[0, 0].set_yticklabels(region_labels)

# -----
# LEFT-BOTTOM: Temperature Change (°C)
# -----
axs[1, 0].set_title("Projected Temperature Change (°C), up to 2030 (A1B)")

```

```

axs[1, 0].axvline(0, color='black', linestyle='--')
axs[1, 0].set_xlim(0, 5)
for i in range(6): #Filtering Through Each Region
    temp_vals = temp_data[i, 1:] # skip first col
    temp_vals = temp_vals[~np.isnan(temp_vals)]
    axs[1, 0].boxplot(
        np.sort(temp_vals),
        positions=[i],
        widths=0.6,
        vert=False,
        patch_artist=True,
        boxprops=dict(facecolor=colors[i], color='black'),
        whiskerprops=dict(color='black'),
        capprops=dict(color='black'),
        medianprops=dict(color='black')
    )

axs[1, 0].set_yticks(range(6))
axs[1, 0].set_yticklabels(region_labels)

axs[0, 1].axis('off') # Hiding old plots code
axs[1, 1].axis('off')

```

```
[30]: (np.float64(0.0), np.float64(1.0), np.float64(0.0), np.float64(1.0))
```

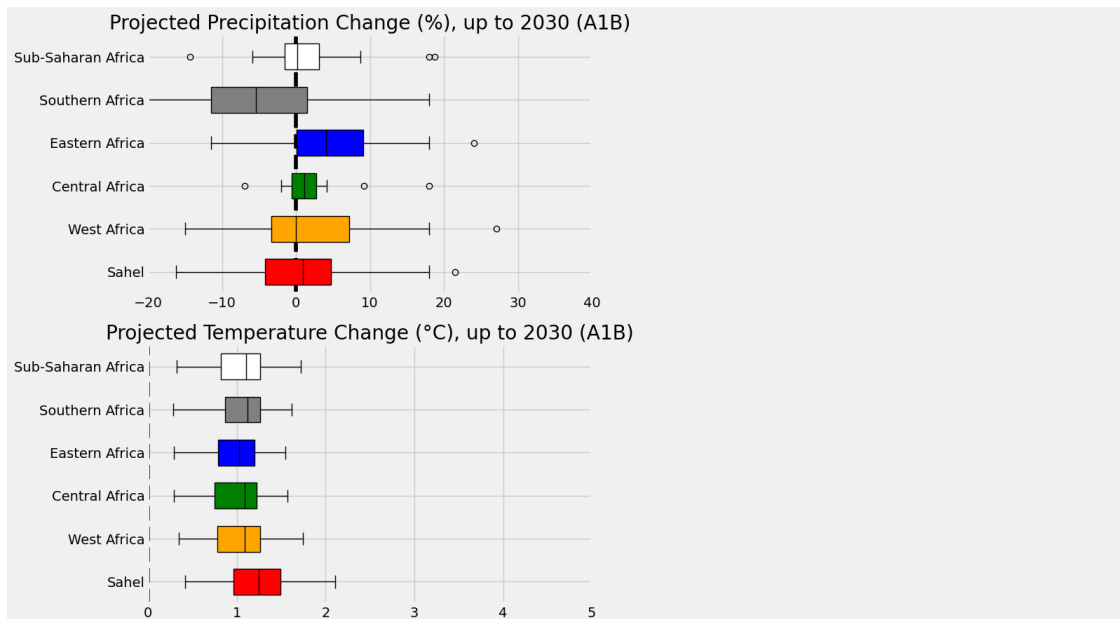


Figure 1 right side


```

[31]: #Figure 1 Right Side Josh

# Load data
# shape: (6, 3, 2, 28)
coef = np.load("bootstrap_1.npy") # shape: (10000, 2)
# for mean precip in column 56

# ReSetup due to us having to replicate the same Figure on multiple computers
region_names = ["Sahel", "West Africa", "Central Africa", "Eastern Africa",
↳ "Southern Africa", "Sub-Saharan Africa"]
region_colors = ['red', 'orange', 'green', 'blue', 'gray', 'white' ]
mean_precip = climconflict.iloc[:, 55].mean() # Was not working without this
↳ baseline, seem slike some inconsistencies within the code.

# Start figure
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10), sharex=True)
axes = axes.flatten()

for i in range(6):
    ax = axes[i] #For loop in each region

    # Extract region-specific projections
    projt_fig1 = clim_full[i, 0, 0, :] # temperature
    projp_fig1 = clim_full[i, 0, 1, :] # precipitation
    projp_scaled_fig1 = projp_fig1 * mean_precip / 100

    # Cleaning up the NAs
    projt_fig1 = projt_fig1[~np.isnan(projt_fig1)]
    projp_scaled_fig1 = projp_scaled_fig1[~np.isnan(projp_scaled_fig1)]

    # (1) Total uncertainty: different proj, different coef
    all_proj_fig1 = np.concatenate([
        projt_fig1[j] * coef[:, 0] + projt_fig1[j] * coef[:, 1] for j in
↳ range(len(projt_fig1)) #Projections Calculation
    ])

    # (2) Regression-only: fixed proj, varying coef
    mean_proj_fig1 = np.median(projt_fig1) #Mean Projection
    confr = mean_proj_fig1 * coef[:, 0] + mean_proj_fig1 * coef[:, 1]

    # (3) Climate-only: varying proj, fixed coef
    med_betas = np.median(coef, axis=0)
    climr = projt_fig1 * med_betas[0] + projt_fig1 * med_betas[1]

    # Assemble all 3 projections
    data_box = [all_proj_fig1 * 100, confr * 100, climr * 100] # percentage
↳ point changes

```

```

labels = ["(1) Both", "(2) Regression", "(3) Climate"]

# Plot boxplots
ax.boxplot(data_box,
            vert=False,
            patch_artist=True,
            labels=labels,
            boxprops=dict(facecolor=region_colors[i], alpha=0.7),
            medianprops=dict(color='black'),
            whiskerprops=dict(color='black'),
            capprops=dict(color='black'),
            flierprops=dict(marker=''))

ax.set_title(region_names[i], fontsize=13)
ax.set_xlim(-5, 20)
ax.grid(True, linestyle="--", alpha=0.4)

# Final layout
fig.suptitle("Figure 1 (Right): Projected Change in Conflict Incidence by_
↳Region", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig("figure1_right_split_panels.png", dpi=300)
plt.show()

```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_box,
```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_box,
```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_box,
```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

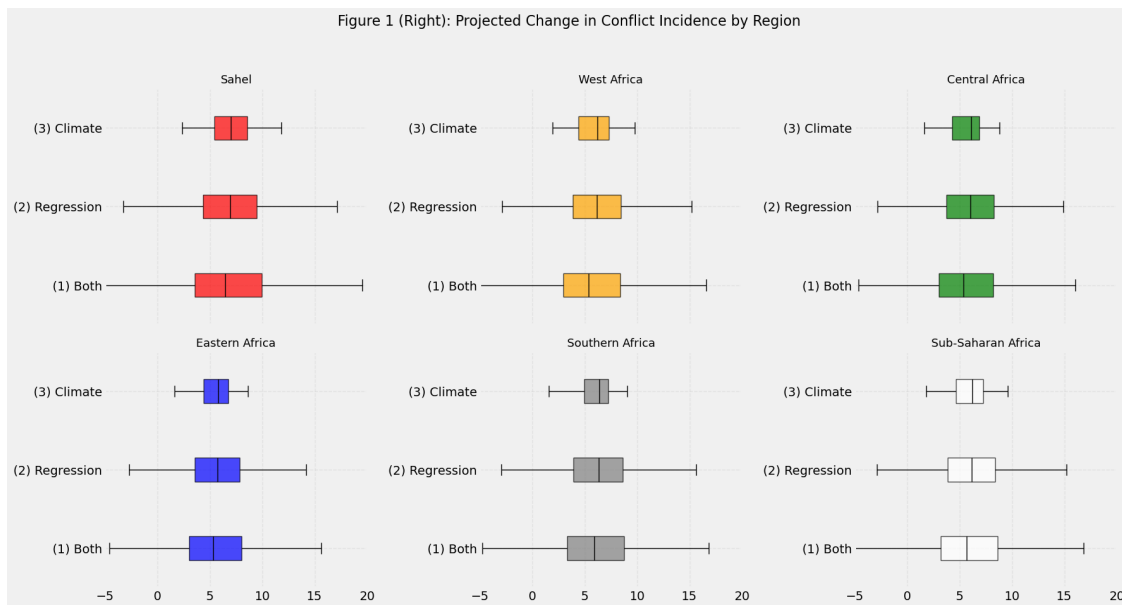
```
ax.boxplot(data_box,
```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_box,
```

/tmp/ipykernel_279/587897468.py:47: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_box,
```



```
[ ]: ## note: translating r code and stata code to python was done with the help AI
      ↳ as it was difficult to ensure consistency and accuracy.
      ## we additionally faced many challenges with our bootstrapping
      ## following r code line by line, code for code, was not possible given the
      ↳ difference in languages, visually our code will look different that the
      ↳ original
      ## replication required an abundant amount of code outside the scope of course
      ↳ content, we utilized new code to help build this replication

      ##thanks for a great class
```